**agent: discovery-backend-modernization-agent cli: Cursor Agent CLI llm: auto run_id: 20260716T142107_dmc0fs generated_at: 2026-07-16T08:51:48.969Z**

# 4. Backend Modernization Hotspots Analysis

**Objective:** Modernize backend architecture and coding practices; strengthen API & integration governance.

**Date:** July 16, 2026 | **Scope:** `shende-shweta/FSDKC` — PHP 8.3 / Laravel 12 (production API) + Node.js 20 / Express 4.21 (dev-api shim)

## Executive Summary

EXECUTIVE SUMMARY

The FSDKC (Klearcom) backend is a dual-runtime stack: a Laravel 12 monolith (`backend/`) backed by Eloquent/MariaDB and MongoDB, plus a parallel Express dev-api (`dev-api/`) that re-implements most REST capabilities against an in-memory store. Architectural layering is immature — only two injectable service classes exist, Eloquent and KPI math live directly in controllers, and 17 of 20 API capabilities are duplicated across both runtimes with no OpenAPI spec, versioning, or contract tests. PHP `extract()` on user input appears in legacy reporting code, and the dev-api exposes unvalidated `req.body` writes plus module-level mutable globals. Overall backend modernization posture is **High Risk**, driven by missing data/service layers, API sprawl, absent API governance, and duplicated business logic across PHP and Node.

| 7 | 3 | 2 | 37 |
|---|---|---|---|
| CONTROLLERS / HANDLERS SCANNED | FILES USING DYNAMIC-VARIABLE PATTERNS | SERVICE CLASSES FOUND | API ENDPOINTS FOUND |

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

**High Risk**

Driven by H3 (21.9% data-layer compliance), H5 (36 inline handlers), H6 (15% single-API capabilities), H7 (0% API governance), and H8 (duplicated cross-runtime logic).

## 4.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---|---|---|---|---|---|---|

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| H1 | Dynamic Variable Creation | Dynamic-var-from-input occurrences | 0 | 1–10 | >10 | 3 `extract()` + unchecked `req.body` in 4 POST handlers | MODE |
| H2 | Global Mutable State | Globals / mutable static state | 0 | 1–5 | >5 | 2 modules ( `store.js` , `mongo.js` ) | MODE |
| H3 | Direct SQL Outside Data Layer | Data-layer compliance % | >90% | 60–90% | <60% | 21.9% (7 of 32 Eloquent calls in services) | HIGH |
| H4 | Static / Singleton Abuse | Business-logic static/singleton classes | 0 | 1–5 | >5 | 0 | GOOD |
| H5 | Missing Service Layer | Handlers with inline business logic | <10 | 10–20 | >20 | 36 (18 Laravel methods + 18 Express routes) | HIGH |
| H6 | API Sprawl | Documented & governed endpoints % | >90% | 80–90% | <80% | 15% single-API capabilities (3 of 20) | HIGH |
| H7 | Missing API Governance | Governance compliance % | 100% | 90–99% | <90% | 0% (no OpenAPI, versioning, or contract tests) | HIGH |
| H8 | Duplicated Cross-Runtime Logic (additional) | Duplicate business-logic blocks across PHP/Node | 0 | 1–3 | >3 | 6 (reachability ×3, `buildTree` ×3) | HIGH |
| H9 | Unvalidated dev-api Inputs (additional) | POST/PUT handlers without schema validation | 0 | 1–3 | >3 | 4 ( `/discovery/jobs` , `/discovery/jobs/:id/start` , `/connect/monitors` , `/connect/monitors/bulk-import` ) | HIGH |

**No additional hotspots beyond the standard set were observed** beyond H8 and H9 documented above.

## 4.2 Hotspot-by-Hotspot Evidence

### H1. Dynamic Variable Creation HIGH

**Benchmark:** `Dynamic-var-from-input occurrences = 3 extract() calls + 4 unchecked req.body POST handlers` → falls in the **Moderate** band (Good 0 · Moderate 1–10 · High Risk >10).

PHP legacy code materializes local variables from raw request arrays via `extract()` , and the Node dev-api assigns request fields directly without DTO validation.

**Example 1** — `backend/app/Legacy/LegacyDataMapper.php:12-18`

```
public function mapReportRow(array $row): array
{
    extract($row, EXTR_SKIP);

    return [
        'label' => $name ?? 'Unknown',
        'metric' => $reachability_pct ?? 0,
```

**Example 2** — `backend/app/Http/Controllers/Api/LegacyReportController.php:20-24`

```
public function carrierSummary(Request $request): JsonResponse
{
    $filters = $request->all();
    extract($filters);

    $monitors = ConnectMonitor::query()
```

**Example 3** — `dev-api/src/server.js:88-97`

```
app.post('/api/discovery/jobs', (req, res) => {
  const job = {
    id: store.nextJobId++,
    name: req.body.name,
    phone_number: req.body.phone_number,
    country_code: req.body.country_code,
```

**Why it matters here:** Any unexpected request field in `carrierSummary` can silently bind to a PHP local variable via `extract()`, shadowing intended symbols and making data flow untraceable. The dev-api mirrors this risk by trusting `req.body` fields without schema checks, including the `bulk-import` endpoint that accepts arbitrary arrays.

**Recommended approach:**

1. Replace `LegacyDataMapper` with typed DTOs ( `CarrierReportRow` , `JobContextDto` ) and explicit field mapping.
2. Refactor `LegacyReportController::carrierSummary` to use `$request->only(['country_code','carrier'])` or a `FormRequest` .
3. Add `zod` / `joi` schemas in `dev-api/src/server.js` for all POST routes.

*No files matched* `backend/**/*.php` *containing* `extract\s*\(` .
*No files matched* `dev-api/**/*.js` *containing* `req\.body\.` .

### H2. Global Mutable State MEDIUM

**Benchmark:** `Globals / mutable static state = 2 modules` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

The dev-api keeps all relational data and Mongo connection handles in module-level mutable singletons shared across every HTTP request.

**Example 1** — `dev-api/src/store.js:3-4`

```
export const store = {
  discoveryJobs: [
```

**Example 2** — `dev-api/src/mongo.js:7-11`

```
let client = null;
let db = null;
let memoryServer = null;
let usingMemory = false;
let dbName = 'klearcom';
```

**Why it matters here:** `store` is mutated in-place by every POST route ( `unshift` , `nextJobId++` ), so concurrent requests can race and cross-contaminate dev sessions. Module-level Mongo handles prevent per-request lifecycle control and complicate test isolation.

**Recommended approach:**

1. Wrap `store` in a factory or request-scoped repository injected into route handlers.
2. Encapsulate Mongo client lifecycle in a `MongoConnection` class with explicit `connect()` / `close()` .
3. Long term, retire the in-memory store once Laravel dev parity is sufficient.

*No files matched* `dev-api/**/*.js` *containing* `export const store` .
*No files matched* `dev-api/**/*.js` *containing* `^let (client|db|memoryServer)` .

### H3. Direct SQL / ORM Outside Data Layer CRITICAL

**Benchmark:** `Data-layer compliance = 21.9%` → falls in the **High Risk** band (Good >90% · Moderate 60–90% · High Risk <60%).

No Repository layer exists. Controllers issue Eloquent queries directly; only `RealTimeTestService` and `MongoService` centralize persistence for async tests and MongoDB.

**Example 1** — `backend/app/Http/Controllers/Api/DashboardController.php:14-18`

```
$discoveryTotal = DiscoveryJob::count();
$discoveryCompleted = DiscoveryJob::where('status', 'completed')->count();
$connectMonitors = ConnectMonitor::count();
$avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
$alerts = ConnectMonitor::where('status', 'alert')->count();
```

**Example 2** — `backend/app/Http/Controllers/Api/LegacyReportController.php:26-38`

```
$monitors = ConnectMonitor::query()
    ->when(isset($country_code), fn ($q) => $q->where('country_code',
$country_code))
    ->when(isset($carrier), fn ($q) => $q->where('carrier', $carrier))
    ->orderByDesc('reachability_pct')
    ->get();
// ...
$recent = ConnectCheckResult::where('connect_monitor_id', $monitor->id)
    ->orderByDesc('checked_at')
    ->limit(20)
    ->get();
```

**Why it matters here:** KPI aggregation, filtering, and reachability math are embedded in HTTP handlers, so the same queries cannot be reused from CLI jobs, queues, or tests without duplicating Eloquent chains. Storage technology changes (MariaDB → read replica, caching) require touching multiple controllers.

**Recommended approach:**

1. Introduce `ConnectMonitorRepository`, `DiscoveryJobRepository`, and `ConnectCheckResultRepository` under `backend/app/Repositories/`.
2. Move dashboard aggregation into `DashboardService` consuming repositories.
3. Relocate legacy report queries from `LegacyReportController` into `LegacyReportService`.

*No files matched `backend/app/Http/Controllers/**/*.php` containing `::(query|where|find|create|count|avg|with|orderBy)`.*

### H4. Static Methods & Singleton Abuse LOW

**Benchmark:** `Business-logic static/singleton classes = 0` → falls in the **Good** band (Good 0 · Moderate 1–5 · High Risk >5).

**Evidence:** Not observed — `MongoService` and `RealTimeTestService` are constructor-injected via Laravel DI; no `static function` business classes or `getInstance()` singletons were found in `backend/`. Controllers do use `app(RealTimeTestService::class)` inside `dispatch()` closures (service-locator pattern), but this is not a static singleton class.

### H5. Missing Service Layer CRITICAL

**Benchmark:** `Handlers with inline business logic = 36` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

Business workflows — KPI math, tree building, reachability scoring, SSE streaming, and CRUD — are implemented inline across six Laravel controllers and the monolithic `dev-api/src/server.js`.

**Example 1** — `backend/app/Http/Controllers/Api/ConnectController.php:56-68`

```
$recentChecks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(20)
```

```
    ->get();

$successRate = $recentChecks->count() > 0
    ? ($recentChecks->where('reachable', true)->count() / $recentChecks-
>count()) * 100
    : 100;

$computedStatus = $successRate < 90 ? 'alert' : 'active';
```

**Example 2** — `backend/app/Http/Controllers/Api/LegacyReportController.php:14-16`

```
/**
 * Fat controller — business logic, DB queries, and KPI math live here (anti-
pattern).
 */
class LegacyReportController extends Controller
```

**Example 3** — `dev-api/src/server.js:56-76` (dashboard KPIs inline in route handler)

```
app.get('/api/dashboard/kpis', async (_req, res) => {
  const discoveryTotal = store.discoveryJobs.length;
  const discoveryCompleted = store.discoveryJobs.filter((j) => j.status ===
'completed').length;
  const avgReach = store.connectMonitors.reduce((s, m) => s +
m.reachability_pct, 0) / store.connectMonitors.length;
```

**Why it matters here:** With only `MongoService` and `RealTimeTestService`, most domain workflows cannot be invoked from Artisan commands, queued jobs, or integration tests without bootstrapping HTTP. The Express shim re-implements the same workflows inline, guaranteeing drift.

**Recommended approach:**

1. Add `DashboardService`, `ConnectService`, `DiscoveryService`, and `LegacyReportService` under `backend/app/Services/`.
2. Reduce controllers to validate → delegate → respond.
3. Decompose `dev-api/src/server.js` into route files + service modules that call shared contracts (or deprecate dev-api).

*No files matched `backend/app/Http/Controllers/Api/*.php`.*

*No files matched `dev-api/src/server.js`.*

### H6. API Sprawl HIGH

**Benchmark:** `Single-API capabilities = 15% (3 of 20)` → falls in the **High Risk** band (Good >90% · Moderate 80–90% · High Risk <80%).

FSDKC exposes 37 route registrations across two stacks implementing the same Klearcom platform. Seventeen capabilities (`/health`, `/dashboard/kpis`, full `/discovery/*` and `/connect/*` trees, Mongo diagnostics) exist in both

Laravel and Express with divergent path params and behavior.

**Example 1** — Laravel `backend/routes/api.php:33-39` vs Express `dev-api/src/server.js:84-86`

```
Route::prefix('discovery')->group(function (): void {
    Route::get('/jobs', [DiscoveryController::class, 'index']);
    Route::post('/jobs', [DiscoveryController::class, 'store']);
```

```
app.get('/api/discovery/jobs', (_req, res) => {
  res.json({ data: store.discoveryJobs });
});
```

**Example 2** — Express-only ungoverned endpoint `dev-api/src/server.js:143-148`

```
app.post('/api/connect/monitors/bulk-import', (req, res) => {
  // No validation, no rate limiting — accepts arbitrary body (security audit
finding)
  const items = Array.isArray(req.body) ? req.body : req.body?.monitors ?? [];
```

**Why it matters here:** Frontend and integrators must guess which runtime is authoritative; bulk-import exists only in dev-api while legacy reports exist only in Laravel. Bug fixes must be applied twice, and response shapes can drift (Eloquent models vs plain JS objects).

**Recommended approach:**

1. Declare Laravel `backend/` as the canonical API; gate dev-api behind `npm run dev` only.
2. Publish a single OpenAPI 3 spec generated from Laravel routes.
3. Remove or proxy duplicate Express routes to Laravel.

*No files matched `dev-api/src/server.js`.*

### H7. Missing API Governance HIGH

**Benchmark:** `Governance compliance = 0%` → falls in the **High Risk** band (Good 100% · Moderate 90–99% · High Risk <90%).

**Evidence:** Not observed — no `openapi.yaml`, Swagger annotations, `apiVersion` prefix, Spectral lint config, or contract test suite exists anywhere in the repository. Routes are defined only in `backend/routes/api.php` and `dev-api/src/server.js`.

### H8. Duplicated Cross-Runtime Logic (additional) HIGH

**Benchmark:** `Duplicate business-logic blocks = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

**Example 1** — Reachability calculation duplicated in `ConnectController.php:56-64`, `RealTimeTestService.php:99-104`, and `dev-api/src/realtime.js:147-152`.

```
const successRate = recent.length > 0
  ? (recent.filter((c) => c.reachable).length / recent.length) * 100
  : 100;
monitor.reachability_pct = Math.round(successRate * 100) / 100;
monitor.status = successRate < 90 ? 'alert' : 'active';
```

**Example 2** — `buildTree()` copy-pasted in `DiscoveryController.php:87-97`, `LegacyReportController.php:79-92`, and `dev-api/src/store.js:57-68`.

```
export function buildTree(nodes, parentId = null) {
  return nodes
    .filter((n) => n.parent_id === parentId)
    .map((n) => ({
      id: n.id,
      children: buildTree(nodes, n.id),
    }));
}
```

**Why it matters here:** Bug fixes to reachability thresholds or tree shape must be applied in three places; the codebase already documents this as "copy-paste debt."

**Recommended approach:** Extract `ReachabilityCalculator` and `DiscoveryTreeBuilder` services in Laravel; delete mirrored implementations from dev-api.

*No files matched* `**/*.{php,js}` *containing* `buildTree`.
*No files matched* `**/*.{php,js}` *containing* `successRate|reachability_pct.*recent`.

### H9. Unvalidated dev-api Inputs (additional) HIGH

**Benchmark:** `POST/PUT handlers without schema validation = 4` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

**Example 1** — `dev-api/src/server.js:143-148` bulk-import accepts arbitrary body with explicit security-debt comment.

**Example 2** — `dev-api/src/server.js:167-173` POST `/connect/monitors` assigns `req.body.name` without null/type checks.

```
app.post('/api/connect/monitors', (req, res) => {
  const monitor = {
    id: store.nextMonitorId++,
    name: req.body.name,
    toll_free_number: req.body.toll_free_number,
    country_code: req.body.country_code,
```

**Why it matters here:** Dev-api is the default `npm run dev` backend; unvalidated writes can corrupt the in-memory store or pass unexpected shapes to Mongo seed paths.

**Recommended approach:** Add centralized validation middleware before route handlers; mirror Laravel FormRequest rules.

*No files matched* `dev-api/**/*.js` *containing* `app\.post\(` .

## 4.3 API & Integration Governance Evidence

### H6. API Sprawl HIGH

See §4.2 H6 for dual-stack duplication evidence. Capability matrix:

| Capability | Laravel | Express dev-api | Governed |
|---|---|---|---|
| Health / Mongo / Dashboard | Yes | Yes | No |
| Discovery CRUD + stream | Yes | Yes | No |
| Connect CRUD + stream | Yes | Yes | No |
| Legacy reports | Yes | No | No |
| Bulk import | No | Yes | No |

### H7. Missing API Governance HIGH

**Benchmark:** `Governance compliance = 0%` → falls in the **High Risk** band.

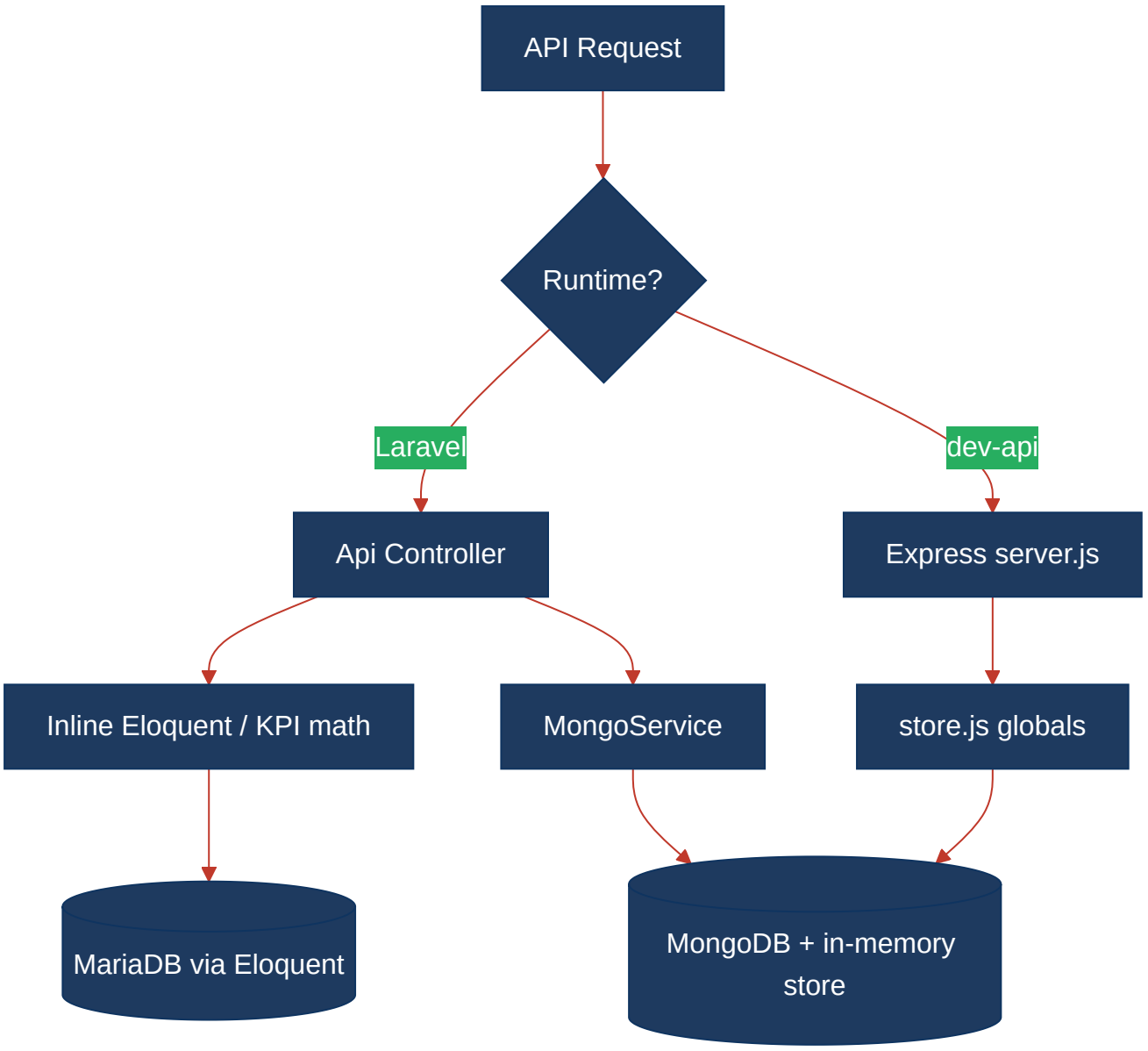
No machine-readable contract, version namespace ( `/v1/...` ), API linting, or consumer-driven contract tests were found. Breaking changes to response shapes (e.g., hardcoded KPI constants `94.2` / `97.8` in `DashboardController` ) ship without detection.

**Recommended approach:**

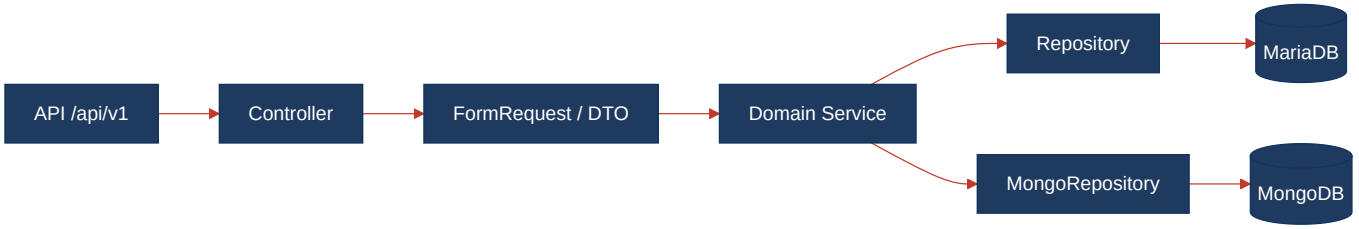
1. Add `docs/openapi.yaml` generated from Laravel route list + FormRequest schemas.
2. Introduce `/api/v1/` version prefix in `backend/routes/api.php` .
3. Add PHPUnit contract tests asserting JSON Schema for each endpoint; add Spectral CI lint.

## 4.4 Diagrams

### Current backend request path




### Modernized service-layer target




### Improvement roadmap

## 4.5 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| H3 Direct SQL Outside Data Layer | Create Repository classes for Connect, Discovery, and CheckResult models; move all Eloquent chains out of controllers | HIGH RISK | CRITICAL |
| H5 Missing Service Layer | Add DashboardService, ConnectService, DiscoveryService, LegacyReportService; slim controllers to delegation only | HIGH RISK | CRITICAL |
| H6 API Sprawl | Deprecate duplicate dev-api routes; document Laravel as canonical API; remove bulk-import shim or port to Laravel with validation | HIGH RISK | HIGH |
| H7 Missing API Governance | Publish OpenAPI 3 spec, add `/api/v1/` versioning, Spectral lint, and PHPUnit contract tests | HIGH RISK | HIGH |
| H8 Duplicated Cross-Runtime Logic (additional) | Extract shared reachability + tree-building into single PHP services; delete mirrored logic in `realtime.js` / `store.js` | HIGH RISK | HIGH |
| H9 Unvalidated dev-api Inputs (additional) | Add Joi/Zod schemas for all dev-api POST/PUT routes; block bulk-import until validated | HIGH RISK | HIGH |
| H1 Dynamic Variable Creation | Remove all `extract()` usage; replace with typed DTOs in LegacyDataMapper and LegacyReportController | MODERATE | HIGH |
| H2 Global Mutable State | Replace `store.js` singleton with injectable repository; encapsulate Mongo globals in connection manager | MODERATE | MEDIUM |

## 4.6 Expected Outcomes

- Repository + service layers raise data-layer compliance above 90% and make domain logic testable without HTTP bootstrapping.
- Retiring duplicate dev-api routes eliminates API sprawl and gives integrators one canonical contract per capability.
- OpenAPI specs with contract tests catch breaking response changes before they reach the React frontend.
- Removing `extract()` and adding DTO validation closes untyped data-flow gaps in legacy reporting and dev-api writes.
- Consolidating reachability and tree-building logic prevents KPI drift between PHP and Node runtimes.